Jeremy Ng Sheng-Wey (20404095)

Title: Developing software for a writing robot

Software Description

The software written uses the C programming language to enable a robot to draw text inputted by a user. This is done by reading text from a user-specified file, mapping the text to corresponding stroke movements based on a given predefined font data sheet. This mapping then generates G-code commands that controls the drawing robot.

The software begins by loading the font data from the file 'SingleStrokeFont.txt'. The name of text file where the text data is extracted from is then requested from the user. This allows the program to load the text input from the text file. Each character within the text input is then processed and its corresponding stroke movements are determined.

The program also requests the user's desired text height to scale the drawn characters to. This is done by calculating a scaling factor within the program to ensure the characters are resized proportionally, allowing the robot to draw text accurately at various sizes. The scaling includes word spacing, line spacing, and proper alignment to ensure the character do not overlap.

The drawn characters are also limited to a 100-unit horizontal constraint. Meaning that the program first pre-calculates the amount of unit space the next word will take. If that space exceeds 100 units, that word is brought down to the next line. The only exception to this case is if the word exceeds 100 units, then it will just take up its own line.

Once all the stroke mapping and scaling are done, the software generates the G-code commands. These G-code commands follow the G-code formatting of the drawing robot. These commands are then sent to the robot via RS232 serial communication, controlling the robot's arm movements. The G-code commands can also be pasted onto an online emulator to simulate the output of the robot, an example of this can be seen in Figure 1.

Intricate error handling is integrated into the software to manage invalid file inputs, incorrect text heights, and other user errors. The modular structure of the program, combined with its flexibility to handle varying input text types and font sizes, makes it's a robust text drawing application.

Figure 1: Sample Output Text via Emulator from G-Code

Project Files

File name	Description
main.c	This file contains the entirety of the main program's code. The main program logic includes input handling, mapping text to its respective don't data, and generating G-code.
rs232.h	Header file that provides declarations for the RS232 serial communication functions, enabling interaction with the drawing robot.
serial.h	Header file that manages the implementation details of serial communication and toggling between the online simulator and robot modes.
rs232.c	Implements RS232 communication, including dummy functions for testing with the simulator.
serial.c	Manages serial communication modes for the simulator and robot testing.
SingleStrokeFont.txt	Font data file given containing stroke movements for each character.
<user-defined>.txt	Text file for testing the system.

Configuration Details:

- To switch between online simulator and robot mode, the #define Serial_Mode line in serial.c should be uncommented and commented respectively.
- User-defined input text file name needs to be entered when prompted when the program runs.

Key Data Items

Name	Data type	Rationale
fontData	fontValue[]	Stores font stroke data for all available characters, extracted from SingleStrokeFont.txt
textData	char[][256]	Holds lines of text input read from the user-specified file.
scaleFactor	float	Determines scaling of text height based on user-input.
currentYOffset	float	Tracks the current vertical position of text at that point in the code.
textFontData	fontValue[]	Stores the mapped strokes for the input text characters.
userHeight	float	Specifies the desired height of the text specified by the user.
gCodeBuffer	Char[]	Temporarily holds the generated G-code commands before it is sent to the robot.

Functions

Loads font data from the SingeStokeFont.txt font data file into the fontData array.

int readFontData(fontValue fontData[])

Parameters:

fontData – array to store font strokes

Reads text input from a user-specified file and stores it in textData array.

*int readTextData(const char *fileName, char textData[][256], int *numLines)*

Parameters:

fileName – name of text data file

textData – array to store user-specified text

numLines – pointer to total number of lines read

Calculates the scaling factor for the height of drawn text based on user input.

*int calculateScaleFactor(float *scaleFactor, float *userHeight)*

Parameters:

scaleFactor – pointer to store calculated scale factor

userHeight – pointer to user-specified text height

Maps user-specified text to its corresponding font data, applying scaling and x and y position offsets.

*int mapTextToFontData(char textData[], fontValue fontData[], fontValue textFontData[], float userHeight, float *currentYOffset)*

Parameters:

textData – array containing text data

fontData – array containing font data

textFontData – array to store font data which have been mapped to the text data

currentYOffset – pointer to the current vertical offset position of drawn text

Generates G-code commands to be sent to the drawing robot

void sendGCode(fontValue textFontData[], float scaleFactor)

Parameters:

textFontData – array to store font data which have been mapped to the text data

scaleFactor – pointer to store calculated scale factor

Sends a line of G-code command to the drawing robot

*void SendCommands(char *buffer)*

Parameters:

buffer – G-code command string

Testing Information

Function	Test Case	Test Data	Expected Output
main()	All functions success	Correct input data for all functions	Outputs correct G-code commands.
	Any function fail	Invalid text height input	Outputs 1 and terminates program.
readFontData()	Valid .txt file	SingleStrokeFont.txt	Font data loaded correctly
	Empty .txt file	EmptyFile.txt	Function prints error message and terminates program
readTextData()	Valid .txt file	Test.txt	Text data loaded correctly
	Empty .txt file	EmptyFile.txt	Function prints error message and terminates program
calculateScaleFactor()	Valid minimum height input	userHeight = 4	Returns scale factor for the input height
	Valid maximum height input	userHeight = 10	Returns scale factor for the input height
	Invalid height input	userHeight = 3	Error message is printed, requests for valid input
	Invalid height input	userHeight = 11	Error message is printed, requests for valid input
mapTextToFontData	Valid .txt file	textData[0] = { h i j }	Font data loaded correctly
	Empty .txt file	textData[0] = { ‘invalid’ }	Function prints error message and terminates program
sendGCode	Valid text font data values	textFontData[0] = { 0 18 1 }	Mapped font data is printed in terminal
	Invalid text font data values	textFontData[0] = {‘invalid’}	Function prints error message and terminates program

Flowchart(s)

Attached along as PDF